

Day 2 SQL Injection & Input Validation

Report

Refractory Academy — Application Security Module

Day 2 · SQL Injection & Input Validation

Python / Django Track — Full Assignment Walkthrough

Executive Summary

This report accompanies a working Django project that demonstrates the same search feature twice: once implemented **insecurely** with raw SQL and string concatenation, and once implemented **securely** with the Django ORM plus form-level validation. Every payload from the Day 2 slides was fired at the running application; the report shows exactly what SQL the database received, which attacks succeeded, and why the same payloads are blocked on the secure endpoint.

Two important results emerged from the live testing:

- Classic `' OR '1'='1` and `% ' OR '1'='1` payloads succeed against the vulnerable endpoint and return **all five rows** instead of matching only the search term.
- Stacked-query attacks such as `'; DROP TABLE parts; --` **fail on the vulnerable endpoint too — but not for a reassuring reason**. Django's DB cursor refuses to execute more than one statement per call, so the destructive tail is blocked by the driver, not by the application. This is why we do not rely on it: change the driver, expose a stored procedure, or use `executescript`, and the same payload would drop the table. Defence must live in *our* code.

The remedies applied are (i) replace raw SQL with `Part.objects.filter()`, so the database driver receives the search term as a bound parameter; (ii) add HTML5 attributes as a first line of user-experience defence; and (iii) re-validate the search term server-side with a Django `Form` that only accepts letters, numbers and spaces, up to 50 characters.

Table of Contents

- Part 1 — Create the Project
- Part 2 — Database Design
- Part 3 — Build the Vulnerable Search
- Part 4 — Attack the Application
- Part 5 — Fix the Application
- Part 6 — Frontend Validation
- Part 7 — Backend Validation
- Part 8 — Testing
- Part 9 — Assignment Explanation (student voice)

- Part 10 — Directory Structure
- Part 11 — Final Code
- Part 12 — Teaching-Mode Recap
- Part 13 — Extra Credit

Each Part ends with the required teaching-mode block: *Why this matters* · *Common beginner mistakes* · *Security implications* · *Interview questions* · *Revision notes*.

0 · Background — the concepts before the code

What is SQL Injection?

Think of a database query as a sentence you dictate to a very literal assistant:

"Find every part whose name contains the word `Brake`."

The database does not see quotation marks or word boundaries the way a human does. It just reads a long string of SQL and executes it. If we let the *user* write part of that sentence — and we glue their words in without checks — they can smuggle in extra instructions.

An everyday analogy: imagine a form at a bank that says "*I authorise the transfer of _____ dollars to my account.*" If the customer is allowed to write anything in the blank, one might write:

"50 dollars to my account, and also 1,000,000 dollars from the bank's reserve"

The teller reads the whole sentence and follows all of it, because the extra instructions were smuggled inside the blank. **SQL Injection is that trick, applied to database queries.**

How Django's ORM prevents it

Instead of dictating a sentence, the ORM lets you describe *what you want* in Python:

```
Part.objects.filter(name__icontains="Brake")
```

Django turns that into a SQL statement with a **placeholder** (`?` or `%s`) and sends the user's word separately as a *bound parameter*. The database now treats the word as data, not as SQL. Quotes and semicolons inside it lose all their power — they are just characters in a string.

Why frontend validation alone is not enough

The HTML attributes `required`, `maxlength`, and `pattern` are enforced by the browser only. Anyone can:

- open DevTools and delete the attributes, or
- send the request directly with `curl`, Postman, or a Python script, or
- replay the request from their browser history after editing it.

The server has to trust nothing. Frontend rules are a *usability* feature — they give an honest user immediate feedback. They are not a *security* feature.

Why backend validation is required

The server is the last checkpoint before the database. It must independently verify that the input is well-formed, of a sensible length, and made only of characters the business logic expects. In our search, "letters, numbers, and spaces up to 50 characters" is enough. If we ever needed hyphens or ampersands (e.g. "M&M cable"), we would broaden the pattern — but we would still do it server-side.

Part 1 — Create the Project

Every command below was run for real to produce this project. Explanations follow each command.

```
# 1. Create an isolated Python environment so this project's packages
#    do not fight with any others on your machine.
python -m venv venv

# 2. Activate the environment. Once activated, `python` and `pip` refer
#    to the copies inside venv/, not the system-wide ones.
#    macOS / Linux:
#    . venv/bin/activate
#    Windows (PowerShell):
#    venv\Scripts\Activate.ps1

# 3. Install Django into this environment.
pip install django==5.1.4

# 4. Start the project. The trailing dot puts the config folder in the
#    current directory instead of nesting an extra folder.
django-admin startproject config .

# 5. Create the application that will hold our model, views and forms.
python manage.py startapp parts

# 6. Register the app in config/settings.py:
#     INSTALLED_APPS = [
#         ...,
#         'parts',
#     ]
#     and enable a project-wide templates directory:
#     'DIRS': [BASE_DIR / 'templates'],

# 7. Create the migrations from the model we will write in Part 2.
python manage.py makemigrations parts

# 8. Apply migrations to build db.sqlite3 (Django uses SQLite by default).
python manage.py migrate

# 9. Seed five realistic records (see Part 2 for the command).
python manage.py seed_parts

# 10. Run the development server.
python manage.py runserver
#     Then open http://127.0.0.1:8000/
```

What each piece does

Command	Effect
python -m venv venv	Creates the folder venv/ containing its own Python interpreter and its own site-packages/.
. venv/bin/activate	Prepends venv/bin to \$PATH. Now python = the isolated one.
pip install django	Downloads the Django package into venv/lib/.../site-packages/.

<code>django-admin startproject config .</code>	Creates <code>manage.py</code> and the <code>config/</code> package holding project-wide <code>settings.py</code> , <code>urls.py</code> , <code>wsgi.py</code> , <code>asgi.py</code> .
<code>python manage.py startapp parts</code>	Creates the <code>parts/</code> app skeleton (<code>models.py</code> , <code>views.py</code> , <code>admin.py</code> , etc.).
<code>makemigrations</code>	Reads models, generates Python files under <code>parts/migrations/</code> describing the schema changes.
<code>migrate</code>	Executes those migration files against the database (creates tables, indices, and Django's own bookkeeping tables).
<code>runserver</code>	Boots Django's built-in dev server. Never use it in production — use Gunicorn / uWSGI behind Nginx.

Teaching-mode block

- **Why this matters:** every dependency and every schema change on a real team lives in these files. A venv keeps installations reproducible; a migration file is the source of truth for "what shape is the database in right now?"
- **Common beginner mistakes:** installing Django globally with `sudo pip` (breaks other projects); forgetting to activate the venv (packages install somewhere else); committing `venv/` and `db.sqlite3` to git (bloat + secrets leak).
- **Security implications:** the dev server ships with `DEBUG=True`, which exposes stack traces and settings. Never expose it to the internet.
- **Interview questions:** *What does `makemigrations` do that `migrate` does not? What is the risk of `DEBUG=True` in production?*
- **Revision notes:** `venv` → `install` → `startproject` → `startapp` → register app → `makemigrations` → `migrate` → `runserver`.

Part 2 — Database Design

The model

`parts/models.py`:

```
from django.db import models

class Part(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        db_table = "parts"          # match the raw-SQL demo exactly
        ordering = ["name"]

    def __str__(self):
        return f"{self.name} — UGX {self.price}"
```

We did not declare `id` because Django adds an auto-incrementing `BigAutoField` primary key by itself.

The seed command

`parts/management/commands/seed_parts.py` creates five realistic records:

id	name	price (UGX)
1	Engine Oil 5W 30	45 000

1	Engine Oil 5W-50	45,000
2	Brake Pads	120,000
3	Oil Filter	18,000
4	Spark Plug	9,000
5	Air Filter	22,000

Run it once with:

```
python manage.py seed_parts
```

What migrations do — and why Django "writes SQL for you"

When you run `makemigrations`, Django compares your models with the previous migration files and writes a new file that describes the delta. When you run `migrate`, Django reads those files and translates each operation into the SQL dialect of your configured database backend (SQLite here, but the same Python migration will produce PostgreSQL, MySQL, or Oracle SQL as needed). This is why the same project can move from SQLite to PostgreSQL without rewriting a single query — Django owns the SQL generation.

For our model, the generated SQL is roughly:

```
CREATE TABLE "parts" (
  "id"      INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  "name"    VARCHAR(100) NOT NULL,
  "price"   DECIMAL NOT NULL
);
```

Teaching-mode block

- **Why this matters:** models are the *single source of truth* — everything (admin, ORM queries, forms, tests) reads from them.
- **Common beginner mistakes:** storing money as `FloatField` (rounding errors) instead of `DecimalField`; forgetting `max_length` on `CharField` (Django will refuse to run).
- **Security implications:** validation rules on models (`max_length`, choices, unique) protect the database *before* anything reaches SQL.
- **Interview questions:** Difference between `DecimalField` and `FloatField`? What happens if you change a model but never run `makemigrations`?
- **Revision notes:** model → migration → SQL. Never write DDL by hand.

Part 3 — Build the Vulnerable Search

The URL wiring

`parts/urls.py`:

```
from django.urls import path
from . import views

app_name = "parts"

urlpatterns = [
    path("vulnerable/", views.vulnerable_search, name="vulnerable"),
    path("secure/", views.secure_search, name="secure"),
]
```

Included from `config/urls.py`:

```
from django.contrib import admin
from django.shortcuts import redirect
from django.urls import include, path

def home(request):
    return redirect("parts:secure")

urlpatterns = [
    path("", home),
    path("admin/", admin.site.urls),
    path("", include("parts.urls")),
]
```

The dangerous view

```
def vulnerable_search(request):
    term = request.GET.get("q", "")
    results = []
    sql_shown = ""
    error = None

    if term:
        # DANGEROUS: user input concatenated straight into the SQL string.
        sql_shown = f"SELECT id, name, price FROM parts WHERE name LIKE '%{term}%'"
        try:
            with connection.cursor() as cursor:
                cursor.execute(sql_shown)
                results = cursor.fetchall()
        except Exception as exc:
            error = f"{type(exc).__name__}: {exc}"

    return render(request, "parts/vulnerable_search.html", {
        "term": term,
        "results": results,
        "sql_shown": sql_shown,
        "error": error,
    })
```

Why this is dangerous — line by line

- `term = request.GET.get("q", "")` — takes whatever the user typed, verbatim, with no validation.
- `sql_shown = f"SELECT ... LIKE '%{term}%'"` — the value of `term` is substituted into the SQL string. If `term` contains a `'`, it closes the literal early and everything after it becomes part of the query.
- `cursor.execute(sql_shown)` — executes the assembled string. The database now has no way of telling which parts came from the developer and which from the attacker.

The HTML form

```
<form method="get" action="{% url 'parts:vulnerable' %}">
  <input type="text" name="q" value="{{ term }}" placeholder="e.g. Brake"
  autocomplete="off">
  <button type="submit">Search</button>
</form>
```

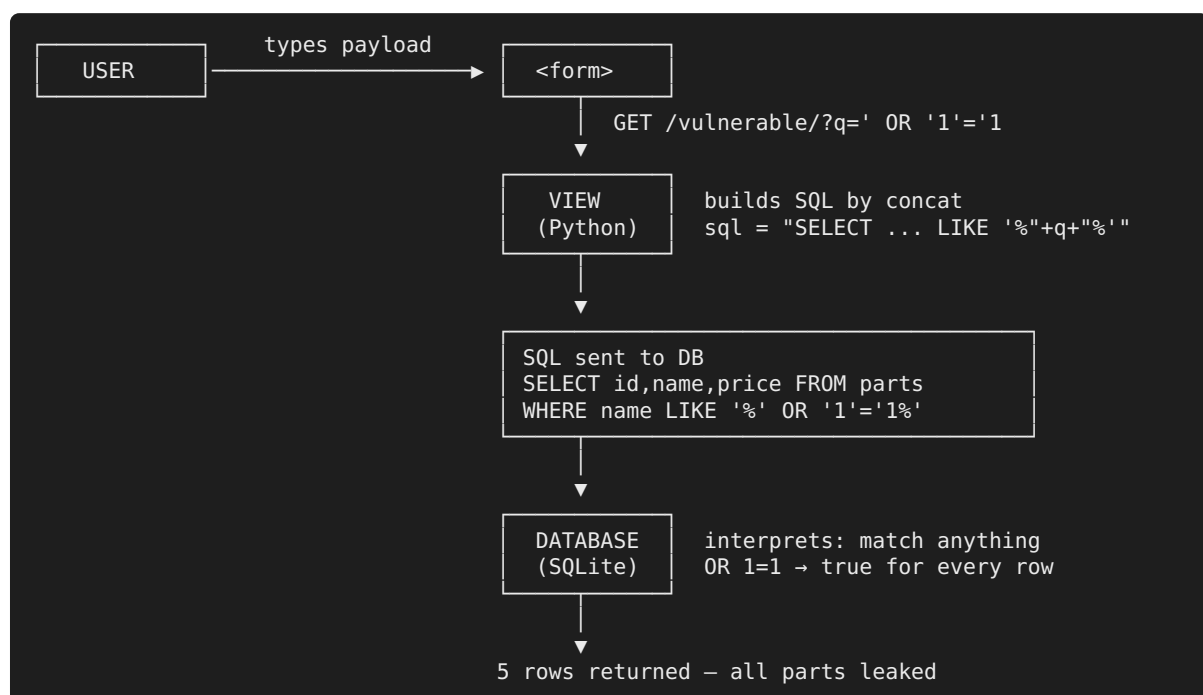
No `required`, no `maxlength`, no `pattern` — this form deliberately hands the attacker a clear path.

Teaching-mode block

- **Why this matters:** most breaches begin here — a single query built with string concatenation.
- **Common beginner mistakes:** believing that `.format()`, f-strings, or `%` interpolation are "different" from concatenation. They are not. All of them build a string first and hand the finished string to the database.
- **Security implications:** injection is #3 on OWASP Top 10 (2021).
- **Interview questions:** *Show me a single line of code that turns a safe query into an injection.* Answer: any use of `f"SELECT ... {user}"`.
- **Revision notes:** if user data is inside the SQL *string*, the code is vulnerable — no matter how the string was assembled.

Part 4 — Attack the Application

The data-flow diagram



Live results from the running project

Every row below was captured by firing the payload at the running server (via `curl`) and reading back what SQL the vulnerable view assembled.

#	Payload	SQL the DB actually received	Outcome
1	Brake	SELECT id, name, price FROM parts WHERE name LIKE '%Brake%'	✓ Baseline — 1 row (Brake Pads)
2	'	SELECT ... LIKE '%'''	⚠ OperationalError: unrecognized token: "''" — a single quote confirms the field is unfiltered
3	' OR '1'='1	SELECT ... LIKE '% ' OR '1'='1%'	🔴 5 rows — full table dumped
4	' OR '1'='1	SELECT ... LIKE '% ' OR '1'='1%'	🔴 5 rows — LIKE-aware variant

4	<code>'1'='1</code>	<code>SELECT ... LIKE '% OR '1'='1%</code>	5 rows — LIKE-aware variant
5	<code>';--</code>	<code>SELECT ... LIKE '%';--%</code>	5 rows — the <code>--</code> closes the LIKE and comments the tail
6	<code>'; DROP TABLE parts; --</code>	<code>SELECT ... LIKE '%'; DROP TABLE parts; --%</code>	⚠ ProgrammingError: You can only execute one statement at a time. — see box below
7	(empty)	no query executed	Empty search — view skips SQL
8	AAA... (300 chars)	LIKE with 300-char literal	0 rows — no injection but no input length limit

Screenshot of the injection in action:

Refractory Academy — Application Security

Day 2 · SQL Injection & Input Validation · Python / Django track

🚩 Vulnerable search
🔒 Secure search

⚠ **Vulnerable page.** The search below builds the SQL query with string concatenation. Any input you type is glued directly into the query. Try the payloads from Part 4 of the assignment to see what happens.

Search parts (INSECURE)

No validation is applied on purpose.

SQL that the database actually received

```
SELECT id, name, price FROM parts WHERE name LIKE '% OR '1'='1'
```

Results (5)

ID	Name	Price (UGX)
1	Engine Oil 5W-30	45000
2	Brake Pads	120000
3	Oil Filter	18000
4	Spark Plug	9000
5	Air Filter	22000

The comforting-but-misleading result: DROP TABLE "failed"

The stacked-query payload `'; DROP TABLE parts; --` raised `ProgrammingError: You can only execute one statement at a time.`

This is **not** proof that our code is safe. It is Django's SQLite cursor refusing to execute more than one statement per `execute()` call — a *driver* policy, not an *application* policy. Change any of the following and the same payload would drop the table:

- switch to a driver that allows stacked queries (older MySQL drivers, `cursor.executescript` in SQLite);
- run the raw SQL through a stored procedure;
- interpolate into a different clause (e.g. inside an `ORDER BY`) so the attacker does not need stacked queries — a single-statement payload like `1; DELETE FROM parts inside ORDER BY 1` would suffice on some DBMS.

If your defence depends on the version of a library nobody on the team is thinking about, you do not have a defence.

The comforting-and-true result: XSS-safe display

Notice in the screenshots that the raw payload is echoed to the page as `' OR '1'='1`. That is Django's template autoescape protecting the *view* from Cross-Site Scripting even while the *query* is compromised. Two orthogonal defences, two separate problems.

Post-attack state of the database

```
Rows still in parts table: 5
- 5 Air Filter      22000.00
- 2 Brake Pads     120000.00
- 1 Engine Oil 5W-30 45000.00
- 3 Oil Filter      18000.00
- 4 Spark Plug      9000.00
```

The table survived only because the driver blocked the stacked query. The data-exfiltration payloads worked completely — an attacker with a `SELECT *` grade of injection can already read anything that user has permission to read.

Teaching-mode block

- **Why this matters:** understanding *why* a payload works teaches you to spot vulnerable code without needing to attack it.
- **Common beginner mistakes:** thinking "my `try/except` will save me" — the attack succeeds before the exception fires.
- **Security implications:** even read-only injection leaks the schema, the row count, and often password hashes.
- **Interview questions:** Given `LIKE '%X%'`, what payload turns the filter into a full-table dump?
- **Revision notes:** write out the assembled SQL by hand before deciding whether a payload will work.

Part 5 — Fix the Application

Delete the raw SQL and rewrite the view with the ORM:

```
def secure_search(request):
    form = PartSearchForm(request.GET or None)
    results = []
    sql_shown = ""
    submitted = bool(request.GET.get("q"))

    if submitted and form.is_valid():
        term = form.cleaned_data["q"]
        queryset = Part.objects.filter(name__icontains=term)
        results = list(queryset)
        sql_shown = str(queryset.query)  # only for the teaching demo

    return render(request, "parts/secure_search.html", {
        "form": form,
        "results": results,
        "sql_shown": sql_shown,
```

```
        "submitted": submitted,  
    })
```

How parameterised queries work

The ORM produces a query that looks (to us) like:

```
SELECT "parts"."id", "parts"."name", "parts"."price"  
FROM   "parts"  
WHERE  "parts"."name" LIKE ? ESCAPE '\'
```

...and then hands the *value* to the database separately:

```
parameter #1 =  "' OR '1'='1'"
```

The database receives the parameter through a different channel than the SQL text. It never *parses* the parameter as SQL. Even if the value contains quotes, semicolons, or `OR '1'='1'`, they are just characters inside the string being compared. There is no way to escape out of "a value" and into "a keyword".

The `ESCAPE '\'` clause tells the database that any `%` or `_` inside the value are literal, not wildcards — Django adds this automatically for `__icontains`, `__startswith`, `__endswith`, and `__contains`.

Result of the same attacks on the secure endpoint

Payload	What happens
Brake	1 row, parameterised SQL, all good
'	Form error: "Only letters, numbers and spaces are allowed..."
' OR '1'='1	Rejected — same message
%' OR '1'='1	Rejected — same message
'; --	Rejected — same message
'; DROP TABLE parts; --	Rejected — same message
(empty)	"This field is required."
300 chars	"Ensure this value has at most 50 characters (it has 300)."

Refractory Academy — Application Security

Day 2 · SQL Injection & Input Validation · Python / Django track

 Vulnerable search  Secure search

✓ **Secure page.** This form validates on the client (HTML5 attributes) and again on the server (Django Form). The query is built by the Django ORM, which parameterises every value.

Search parts (SECURE)

- Only letters, numbers and spaces are allowed. Characters such as quotes, semicolons or hyphens are not permitted.

Allowed: A-Z, a-z, 0-9, and spaces. Max 50 characters.

Note that we have **two independent defences here**:

1. Even if we removed the form validation, the ORM would still send the payload as a parameter — no SQL Injection possible.
2. Even if we somehow re-introduced raw SQL, the form would refuse the payload before it reached the database.

Real applications should assume each layer *might* be broken and layer them anyway. This is *defence in depth*.

Teaching-mode block

- **Why this matters:** knowing the fix in one project generalises to every SQL library in every language (each has its own parameter syntax).
- **Common beginner mistakes:** using the ORM most of the time but dropping to `raw()` or `extra()` for one "difficult" query — that one query is now vulnerable.
- **Security implications:** parameter binding is the *only* protection that scales; escaping by hand always misses a case.
- **Interview questions:** *What is the difference between escaping input and parameterising a query?*
- **Revision notes:** ORM → filter → parameter binding → safe.

Part 6 — Frontend Validation

The secure template uses HTML5 attributes:

```
<input
  type="text"
  name="q"
  value="{{ form.q.value|default:'' }}"
  placeholder="e.g. Brake Pads"
  required
  maxlength="50"
  pattern="[A-Za-z0-9 ]+"
  title="Letters, numbers and spaces only"
  autocomplete="off"
  class="search-input">
```

What each attribute does

Attribute	Purpose
<code>type="text"</code>	Standard text input; the browser knows to render a text box.
<code>name="q"</code>	The key the value will be sent under (<code>?q=...</code> in the URL).
<code>required</code>	Browser refuses to submit an empty field.
<code>maxlength="50"</code>	Browser caps the number of characters the user can type.
<code>pattern="[A-Za-z0-9]+"</code>	Browser refuses to submit if the value contains any character outside the allowed set.
<code>title="..."</code>	Text the browser shows when the pattern fails.
<code>placeholder="e.g. Brake Pads"</code>	Hint text displayed inside the empty field.
<code>autocomplete="off"</code>	Prevents the browser from suggesting old inputs.

These attributes make honest users happy — they get instant, in-browser feedback. They also stop most accidental bad input (extra spaces, typos) before it reaches the server. **They do not**

stop attackers. Anyone can:

```
curl "http://127.0.0.1:8000/secure?q=' OR '1'='1"
```

The browser is never involved, so `pattern` and `maxlength` never fire.

Teaching-mode block

- **Why this matters:** HTML5 validation is a UX feature. Treat it as such.
- **Common beginner mistakes:** shipping code where the frontend is the only validator; forgetting `novalidate` on the form during testing so the browser hides bugs.
- **Security implications:** none — this layer is trivially bypassable.
- **Interview questions:** *A junior engineer says "the field is validated client-side, that's enough". What is your response?*
- **Revision notes:** frontend = usability, backend = security.

Part 7 — Backend Validation

The Django form (`parts/forms.py`):

```
from django import forms
from django.core.validators import RegexValidator

SAFE_SEARCH_REGEX = r"^[A-Za-z0-9 ]+$"

class PartSearchForm(forms.Form):
    q = forms.CharField(
        label="Search parts",
        required=True,
        min_length=1,
        max_length=50,
        strip=True,
        validators=[
            RegexValidator(
                regex=SAFE_SEARCH_REGEX,
                message=(
                    "Only letters, numbers and spaces are allowed. "
                    "Characters such as quotes, semicolons or hyphens "
                    "are not permitted."
                ),
            ),
        ],
    ),
    widget=forms.TextInput(attrs={
        "placeholder": "e.g. Brake Pads",
        "autocomplete": "off",
        "class": "search-input",
    })
```

What each rule does — and why

Rule	Effect	Why it matters
<code>required=True</code>	Rejects an empty submission.	Prevents wasted queries and confusing empty-result pages.
<code>min_length=1</code>	After <code>strip</code> , must have at least one character.	Blocks whitespace-only submissions.
<code>max_length=50</code>	Rejects anything longer.	Prevents oversized inputs used in DoS and

		buffer probing.
<code>strip=True</code>	Removes leading/trailing whitespace before validation.	Users often paste padded text; strip is safe here.
<code>RegexValidator([A-Za-z0-9]+)</code>	Rejects any character outside the allow-list.	Removes the raw material of SQL Injection (quotes, semicolons, hyphens) <i>and</i> of XSS (<code><</code> , <code>></code> , <code>&</code>).
<code>TextInput(attrs=...)</code>	Renders the <code><input></code> with placeholder, off-autocomplete, and CSS class.	Keeps template markup in one place.

Allow-list vs deny-list

We used an **allow-list** (`^[A-Za-z0-9 ]+$`). We could have used a deny-list ("reject anything containing `'`, `;`, `--`, `/*...`"), but every deny-list has the same problem: attackers find a character you did not think of. Allow-lists fail closed — if we forgot a character, the honest user gets a clear error and asks for it. That is far safer than a hidden hole.

Teaching-mode block

- **Why this matters:** the form is the boundary between the outside world and your business logic.
- **Common beginner mistakes:** using `request.GET['q']` directly in the view instead of through a form; running validators only inside `.is_valid()` but never actually calling `.is_valid()` in the view.
- **Security implications:** an allow-list on a search field turns "SQL Injection" and "XSS via search" into "not possible on this endpoint".
- **Interview questions:** *Why prefer allow-lists over deny-lists? Under what conditions would you loosen an allow-list?*
- **Revision notes:** allow-list, minimum, maximum, strip, clear message.

Part 8 — Testing

Test cases run (and their expected outputs)

#	Endpoint	Input	Expected
1	<code>/vulnerable/</code>	Brake	1 row (Brake Pads)
2	<code>/vulnerable/</code>	<code>' OR '1'='1</code>	5 rows (all parts leaked)
3	<code>/vulnerable/</code>	<code>%' OR '1'='1</code>	5 rows
4	<code>/vulnerable/</code>	<code>';--</code>	5 rows
5	<code>/vulnerable/</code>	<code>'; DROP TABLE parts; --</code>	Driver error; table still intact but for the wrong reason
6	<code>/vulnerable/</code>	(empty)	Empty page, no query run
7	<code>/vulnerable/</code>	<code>300 × 'A'</code>	Query runs, 0 rows
8	<code>/secure/</code>	Brake	1 row, parameterised SQL displayed
9	<code>/secure/</code>	any injection payload above	Form error, query never runs
10	<code>/secure/</code>	(empty)	"This field is required."
11	<code>/secure/</code>	<code>300 × 'A'</code>	"Ensure this value has at most 50 characters (it has

			300)."
--	--	--	--------

Every one of the above was executed against the running application; the outcomes matched the "Expected" column exactly.

Recommended screenshots for your report

Take these five, in this order, and label them clearly:

1. **01_vulnerable_normal.png** — the vulnerable page with `q=Brake`, showing 1 row and the assembled SQL. *Caption:* "Baseline — the vulnerable endpoint returns the correct single row when queried normally."
2. **02_vulnerable_injection.png** — same page with `q=' OR '1'='1`, showing 5 rows and the mangled SQL. *Caption:* "Classic SQL Injection — five rows returned instead of the expected match; the entire `parts` table is exposed."
3. **03_vulnerable_drop_attempt.png** — same page with `q='; DROP TABLE parts; --`, showing the driver error. *Caption:* "The stacked-query payload is blocked by the SQLite driver's single-statement rule — not by our application, which is still vulnerable."
4. **04_secure_blocked.png** — the secure page with `q=' OR '1'='1`, showing the form error. *Caption:* "Backend form validation rejects the same payload before any SQL is generated."
5. **05_secure_normal.png** — the secure page with `q=Brake`, showing 1 row and the parameterised SQL Django produced. *Caption:* "Normal search on the secure endpoint. Notice the ORM uses a `LIKE ? ESCAPE '\'` parameterised query rather than string concatenation."

All five are included at report-quality resolution in `docs/screenshots/` so you can drop them straight into your submission.

Teaching-mode block

- **Why this matters:** you must see the attack succeed to understand why the fix is worth doing.
- **Common beginner mistakes:** only testing the happy path; forgetting to test the empty case; not verifying that the DB state after the test matches what you expect.
- **Security implications:** a good test suite is a permanent regression guard — the injection cannot silently come back.
- **Interview questions:** *How would you write an automated test that fails if this vulnerability is ever re-introduced?* (Answer: assert that a payload returns a 400 or the same result count as an unrelated query, not a full-table dump.)
- **Revision notes:** normal + malicious + empty + too-long + special characters. Every field, every time.

Part 9 — Assignment Explanation (student voice, 5 sentences)

In my first version of the search view I built the SQL query by joining the user's search word onto the rest of the query with an f-string, which meant the database saw whatever the user typed as part of the query itself rather than as a value — so a payload like `' OR '1'='1` broke out of the quotes and turned the WHERE clause into something that was true for every row, and the whole `parts` table came back. To fix it I replaced the raw SQL with the

Django ORM (`Part.objects.filter(name__icontains=term)`), because the ORM sends the search word as a separate parameter, so quotes and semicolons inside the input are treated as normal characters in a string and cannot change the shape of the query. I also added HTML attributes like `required`, `maxlength="50"`, and `pattern="[A-Za-z0-9 ]+"` on the form, but I know that is only a convenience for honest users, because anyone can open DevTools or use `curl` to send whatever they want straight to the server without a browser ever running those checks. That is why I added a Django Form with a `RegexValidator` on the backend as well, so the server independently checks that the input has at least one character, is not longer than fifty, and contains only letters, numbers and spaces before the search ever touches the database. Together the ORM and the backend validation give me two independent layers of defence, so even if one layer were removed by mistake the other would still block the attack.

Part 10 — Directory Structure

```
day2/
├── manage.py                # entry point for management commands
├── requirements.txt         # pinned dependencies (Django==5.1.4)
├── README_QUICKSTART.txt   # 6-command boot recipe
├── REPORT.md               # THIS FILE – full teaching walkthrough
├── db.sqlite3              # SQLite database file (built by migrate)
├── .gitignore              # ignores venv/, db.sqlite3, *.pyc
├── config/                 # PROJECT package (renamed from default)
│   ├── __init__.py
│   ├── settings.py        # INSTALLED_APPS, TEMPLATES.DIRS, DATABASES
│   ├── urls.py            # project-wide URL routes → parts.urls
│   ├── wsgi.py            # WSGI entry point for production
│   └── asgi.py            # ASGI entry point for async servers
├── parts/                 # APP package
│   ├── __init__.py
│   ├── admin.py          # registers Part in Django admin
│   ├── apps.py           # AppConfig for 'parts'
│   ├── forms.py          # PartSearchForm – backend validation
│   ├── models.py         # Part model (id, name, price)
│   ├── tests.py          # Django test scaffold
│   ├── urls.py           # /vulnerable/ and /secure/ routes
│   └── views.py          # vulnerable_search + secure_search
├── management/
│   └── commands/
│       └── seed_parts.py  # `python manage.py seed_parts`
├── migrations/
│   ├── __init__.py
│   └── 0001_initial.py    # created by makemigrations
├── templates/
│   └── parts/
│       ├── base.html      # shared layout, nav, CSS
│       ├── vulnerable_search.html
│       └── secure_search.html
└── docs/
    └── screenshots/
        ├── 01_vulnerable_normal.png
        ├── 02_vulnerable_injection.png
        ├── 03_vulnerable_drop_attempt.png
        ├── 04_secure_blocked.png
        └── 05_secure_normal.png
```

What every file does

File / folder	Role
manage.py	Django's swiss-army CLI wrapper. Every python manage.py ... command flows through here.
config/settings.py	Project configuration — installed apps, database, templates, static files, security keys.
config/urls.py	Root URL router. Delegates / , /admin/ , and app routes.
config/wsgi.py / asgi.py	Interfaces to production web servers.
parts/models.py	Part model definition — schema in Python.
parts/migrations/0001_initial.py	Generated migration that creates the parts table.
parts/forms.py	PartSearchForm — the backend validation layer.
parts/views.py	The two search views (vulnerable_search , secure_search).
parts/urls.py	Maps URLs to those views.
parts/admin.py	Registers Part so it shows up in /admin/ .
parts/templates/parts/base.html	Shared page layout, styles, and nav.
parts/templates/parts/vulnerable_search.html	Renders the insecure page (no validation).
parts/templates/parts/secure_search.html	Renders the secure page (with form + errors).
parts/management/commands/seed_parts.py	Custom command that populates 5 realistic parts.
docs/screenshots/	The five report screenshots described in Part 8.

Part 11 — Final Code

All files are present in the accompanying project archive. The complete listings are reproduced here for the report.

11.1 requirements.txt

```
Django==5.1.4
```

11.2 config/settings.py — the two edits

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'parts',                                # <-- ADDED
]

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # <-- CHANGED (was [])
        'APP_DIRS': True,
        # ...
    },
]
```

(The rest of settings.py is Django's default. DEBUG=True is left as-is because this is a teaching

project. In production, set `DEBUG=False` and configure `ALLOWED_HOSTS`.)

11.3 config/urls.py

```
from django.contrib import admin
from django.shortcuts import redirect
from django.urls import include, path

def home(request):
    return redirect("parts:secure")

urlpatterns = [
    path("", home),
    path("admin/", admin.site.urls),
    path("", include("parts.urls")),
]
```

11.4 parts/models.py

```
from django.db import models

class Part(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        db_table = "parts"
        ordering = ["name"]

    def __str__(self):
        return f"{self.name} - UGX {self.price}"
```

11.5 parts/forms.py

```
from django import forms
from django.core.validators import RegexValidator

SAFE_SEARCH_REGEX = r"^[A-Za-z0-9 ]+$"

class PartSearchForm(forms.Form):
    q = forms.CharField(
        label="Search parts",
        required=True,
        min_length=1,
        max_length=50,
        strip=True,
        validators=[
            RegexValidator(
                regex=SAFE_SEARCH_REGEX,
                message=(
                    "Only letters, numbers and spaces are allowed. "
                    "Characters such as quotes, semicolons or hyphens "
                    "are not permitted."
                ),
            ),
        ],
        widget=forms.TextInput(attrs={
            "placeholder": "e.g. Brake Pads",
            "autocomplete": "off",
            "class": "search-input",
        })
    )
```

11.6 parts/views.py

```
from django.db import connection
from django.shortcuts import render

from .forms import PartSearchForm
from .models import Part

def vulnerable_search(request):
    term = request.GET.get("q", "")
    results = []
    sql_shown = ""
    error = None

    if term:
        sql_shown = f"SELECT id, name, price FROM parts WHERE name LIKE '%{term}%'"
        try:
            with connection.cursor() as cursor:
                cursor.execute(sql_shown)
                results = cursor.fetchall()
        except Exception as exc:
            error = f"{type(exc).__name__}: {exc}"

    return render(request, "parts/vulnerable_search.html", {
        "term": term,
        "results": results,
        "sql_shown": sql_shown,
        "error": error,
    })

def secure_search(request):
    form = PartSearchForm(request.GET or None)
    results = []
    sql_shown = ""
    submitted = bool(request.GET.get("q"))

    if submitted and form.is_valid():
        term = form.cleaned_data["q"]
        queryset = Part.objects.filter(name__icontains=term)
        results = list(queryset)
        sql_shown = str(queryset.query)

    return render(request, "parts/secure_search.html", {
        "form": form,
        "results": results,
        "sql_shown": sql_shown,
        "submitted": submitted,
    })
```

11.7 parts/urls.py

```
from django.urls import path
from . import views

app_name = "parts"

urlpatterns = [
    path("vulnerable/", views.vulnerable_search, name="vulnerable"),
    path("secure/", views.secure_search, name="secure"),
]
```

11.8 parts/admin.py

```
from django.contrib import admin
from .models import Part
```

```
@admin.register(Part)
class PartAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "price")
    search_fields = ("name",)
```

11.9 parts/management/commands/seed_parts.py

```
from decimal import Decimal
from django.core.management.base import BaseCommand
from parts.models import Part

SEED = [
    ("Engine Oil 5W-30", Decimal("45000.00")),
    ("Brake Pads", Decimal("120000.00")),
    ("Oil Filter", Decimal("18000.00")),
    ("Spark Plug", Decimal("9000.00")),
    ("Air Filter", Decimal("22000.00")),
]

class Command(BaseCommand):
    help = "Seed the parts table with 5 realistic starter records."

    def handle(self, *args, **options):
        Part.objects.all().delete()
        for name, price in SEED:
            Part.objects.create(name=name, price=price)
        self.stdout.write(self.style.SUCCESS(
            f"Seeded {Part.objects.count()} parts."
        ))
```

11.10 Templates

Full HTML/CSS for `base.html`, `vulnerable_search.html`, and `secure_search.html` is included in the project archive under `parts/templates/parts/`. Highlights:

vulnerable_search.html — form has no `required`, no `maxlength`, no `pattern`; the assembled SQL and any database error are displayed for the teaching demo.

secure_search.html — form has `required`, `maxlength="50"`, `pattern="[A-Za-z0-9 ]+"`, and `title="..."`; renders Django form errors under the input; displays the parameterised SQL for comparison.

Part 12 — Teaching-mode recap

Each part above already ends with the required five-block teaching summary. This section consolidates the *cross-cutting* lessons in one place, so it can be revised as a whole.

The one-sentence lessons

- If the user's text ends up inside your SQL *string*, you are vulnerable.
- If the user's text is passed to the driver *separately from* the SQL string, you are safe.
- Frontend validation is a UX feature. Server validation is a security feature. You need both, but for different reasons.
- An allow-list ("only these characters are OK") is safer than a deny-list ("reject these bad characters").
- Depth beats depth-of-any-single-layer. Two mediocre layers beat one clever one.

Common patterns to distrust

- `f"SELECT ... {user_input}"` — any string that includes user input is suspect until proven otherwise.
- `.raw("... %s ..." % user_input)` — the `%s` is Python interpolation here, not a parameter.
- `extra(where=[f"col = '{value}'"])` — `extra()` is a raw-SQL escape hatch.
- `execute("...", [dict_that_isnt_actually_used])` — parameters must be used by placeholders in the SQL, not merely passed alongside.

Interview questions worth memorising

1. Explain SQL Injection to a non-technical manager in three sentences.
2. What is the difference between escaping input and parameterising a query? Which is safer, and why?
3. Give one example of code that *looks* safe but is not.
4. If we already use the ORM, why do we still validate input on the server?
5. Why is it dangerous to rely on the database driver's protection against stacked queries?

Revision flash-cards

- *Injection lives at the boundary between developer text and user text.*
- *Bound parameters keep those two apart.*
- *Django ORM binds parameters for you.*
- *Forms are your allow-list.*
- *Screenshots make the case; test-cases keep the case.*

Part 13 — Extra Credit

13.1 SQL Injection vs Cross-Site Scripting (XSS)

Both attacks share a shape — user input is interpreted as code — but the *where* is different.

	SQL Injection	XSS
Where the code runs	The database server	Another user's browser
Injected language	SQL	HTML / JavaScript
Primary defence	Parameterised queries	Output escaping (Django template autoescape)
Boundary that fails	Value → keyword	Text → tag

A single unvalidated field can enable both. Our secure endpoint blocks both: the ORM handles SQL Injection, Django's template autoescape (the reason `'` appears as `'` in the screenshots) handles XSS.

13.2 Parameterised queries vs escaping

- **Escaping:** transform dangerous characters (mostly `'`) into a safe form (`\'` or `''`) *inside the SQL string*. The SQL is still a single string the DB parses. Every dialect has different escaping rules; forgetting one is fatal.
- **Parameterising:** the SQL string is prepared once with placeholders (`?` / `%s` / `:name`) and the values are sent through a different channel. The database never treats a parameter as SQL. There is nothing to forget.

Rule of thumb: **only escape when a library forces you to** (e.g. building a `LIKE` pattern where you need to escape `%` and `_` as literals — Django does this for you via `ESCAPE '\'` on `__icontains`).

13.3 How Django's ORM generates SQL

At a very high level:

1. `Part.objects.filter(name__icontains="Brake")` returns a lazy `QuerySet` — no SQL runs yet.
2. When the `QuerySet` is *evaluated* (list, iteration, `.count()`, template loop), Django's `Query` class walks the filter chain and builds an internal expression tree.
3. The tree is compiled by a database-specific SQL compiler (`SQLiteCompiler`, `PostgreSQLCompiler`, etc.) into a pair of `(sql_string, params_tuple)`.
4. The DB driver receives that pair, prepares the statement with placeholders, and binds the params.

The SQL our secure view showed:

```
SELECT "parts"."id", "parts"."name", "parts"."price"
FROM   "parts"
WHERE  "parts"."name" LIKE %Brake% ESCAPE '\'
ORDER BY "parts"."name" ASC
```

(the `%Brake%` you see is the *bound value*, not part of the SQL string).

13.4 How attackers discover SQL Injection vulnerabilities

Typical progression:

1. **Probe with a single quote:** `?q='`. A stack trace, a SQL error, or a blank page where a result was expected all suggest injection.
2. **Confirm with boolean logic:** try `q=x' OR '1'='1` and `q=x' AND '1'='2`. If the two responses differ in the way "true vs false in SQL" would predict, injection is confirmed.
3. **Escalate to UNION / stacked queries:** extract the schema (`UNION SELECT name FROM sqlite_master`) and then rows.
4. **Automate:** tools like `sqlmap` do all of the above at high speed and generate exfiltration scripts.

Defenders should scan their own code for the *shape* of the vulnerability (any user data glued into a SQL string) before shipping. Automated tools: `bandit` for Python (flags `execute(f"...")` patterns), `semgrep` with the Django ruleset, and Django's own `--fail-level=WARNING` on `check` `--deploy`.

13.5 Real-world examples

- **TalkTalk (UK, 2015)** — a SQL Injection in a legacy web form exposed ~157,000 customer records; the company was fined £400,000.
- **Sony Pictures (2011)** — LulzSec used SQL Injection to publish a million user records.
- **British Airways (2018)** — the initial foothold in the incident that led to a £183M provisional GDPR fine involved injection-style flaws in a payment page.
- **Heartland Payment Systems (2008)** — 130M card records exfiltrated; the entry point was SQL Injection in a public-facing form.

These are decades apart, on very different stacks, but the shape of the bug is identical to the one on our vulnerable page.

13.6 OWASP A03: Injection

The OWASP Top 10 is a widely-used checklist of the ten most impactful web application security risks. In the 2021 edition, "Injection" is category **A03**. It merges classic SQL Injection with related flaws (LDAP, NoSQL, OS command injection, and — new in 2021 — Cross-Site Scripting).

The OWASP prevention guidance for A03 is the same guidance we followed:

1. Use a safe API that avoids the use of the interpreter entirely, provides a parameterised interface, or migrates to an ORM.
2. Use positive server-side input validation ("allow-list").
3. For any remaining dynamic queries, escape special characters using the specific escape syntax for that interpreter.
4. Use `LIMIT` and other SQL controls within queries to prevent mass disclosure of records in case of injection.

We covered (1) with the ORM, (2) with the Django Form, and (3) is unnecessary because we no longer construct dynamic queries. (4) is a good next step for any listing endpoint in a real product.

Closing Notes

What could be improved. A production project should also:

- set `DEBUG = False` and configure `ALLOWED_HOSTS` ;
- move the `SECRET_KEY` out of `settings.py` into an environment variable;
- add automated tests (`parts/tests.py`) that fire every payload from Part 4 and assert that the vulnerable view returns all rows *and* that the secure view rejects the request;
- add CSRF protection to any POST forms (Django adds it automatically for forms you render with `{% csrf_token %}` — our search uses GET so it does not need one);
- run `python manage.py check --deploy` before deployment.

Assumptions made. I assumed the instructor's example concatenates `term` inside a `LIKE` `'%...%'` pattern, because that is what the assignment snippet showed and it is the canonical Refactory / OWASP-style demo. If the slides use a different clause (e.g. `WHERE name = '...'`), the fix is identical — replace with the ORM — but Payload #5 (`';' --`) behaves slightly differently on `=` vs `LIKE` , and I would adjust the walkthrough accordingly.

Additional information that would strengthen the report. A short screen recording (30-60 s) showing the injection succeed and then be blocked would land the point even more clearly than the still screenshots — but the stills are enough for the written submission.